

Lecture ⑤

Page ⑤

estimated cost

$$f(n) = g(n) + h(n) : A^* \text{ Algorithm depends on } f(n)$$

Real Cost

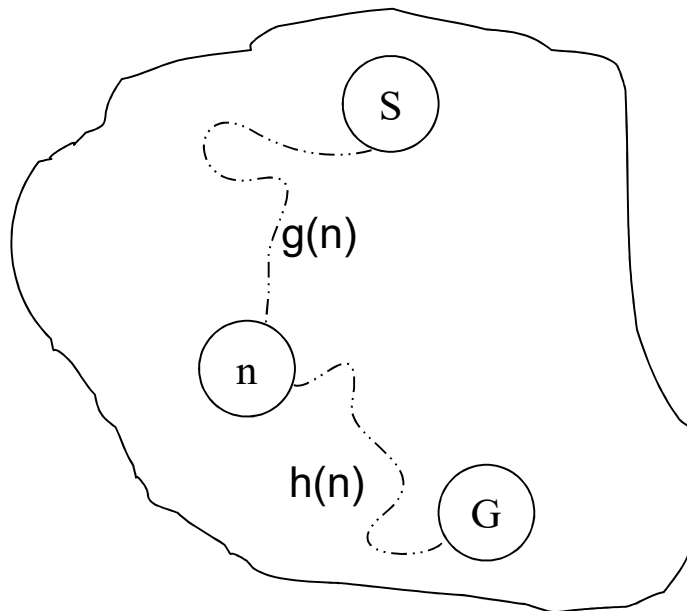
if we have h_1 & h_2 any two admissible functions
then $h(n) = \max\{h_1(n), h_2(n)\}$ is also admissible

Properties of A^*

- 1- Complete : Yes.
- 2- Time Complexity : EXponential.
- 3- Space Complexity : EXponential.
- 4- Optimality : Yes.

Algorithm A*

- One of the most important advances in AI search algs.
- **Idea**: avoid expanding paths that are already expensive
 $f(n) = g(n) + h(n)$
- $g(n)$ = least cost path to n from S found so far
- $h(n)$ = estimated cost to goal from n
- $f(n)$ = estimated total cost of path through n to goal



The A* procedure

Hill-climbing (and its improved versions) may miss an optimal solution. Here is a search method that ensures **optimality** of the solution.

The algorithm

keep a list of partial paths (initially root to root, length 0);

repeat

succeed if the first path P reaches the goal node;

otherwise remove path P from the list;

extend P in all possible ways, add new paths to the list;

sort the list by **the sum of two values**: the real cost of P till now,
and an **estimate** of the remaining distance;

prune the list by leaving only the shortest path for each node
reached so far;

until

success or the list of paths becomes empty;

The A^* procedure

A heuristic that never overestimates is also called **optimistic** or **admissible**.

We consider three functions with values ≥ 0 :

- $g(n)$ is the actual cost of reaching node n ,
- $h^*(n)$ is the actual *unknown* remaining cost,
- $h(n)$ is the optimistic estimate of $h(n)$.

Admissible heuristics

- A heuristic $h(n)$ is **admissible** if for every node n , $h(n) \leq h^*(n)$, where $h^*(n)$ is the **true** cost to reach the goal state from n .
- An admissible heuristic **never overestimates** the cost to reach the goal, i.e., it is **optimistic**
- **Theorem**: If $h(n)$ is admissible, A^* using is optimal

Admissible heuristics

E.g., for the 8-puzzle:

- $h_1(n)$ = number of misplaced tiles
- $h_2(n)$ = total Manhattan distance

(i.e., no. of squares from desired location of each tile)

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- $h_1(S) = ?$
- $h_2(S) = ?$

Admissible heuristics

E.g., for the 8-puzzle:

- $h_1(n)$ = number of misplaced tiles
- $h_2(n)$ = total Manhattan distance

(i.e., no. of squares from desired location of each tile)

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- $h_1(S) = ?$ 8
- $h_2(S) = ?$ $3+1+2+2+2+3+3+2 = 18$

Admissible heuristics

- If $h_2(n) \geq h_1(n)$ for all n , both are admissible
- Then h_2 dominates h_1 and is usually better for search

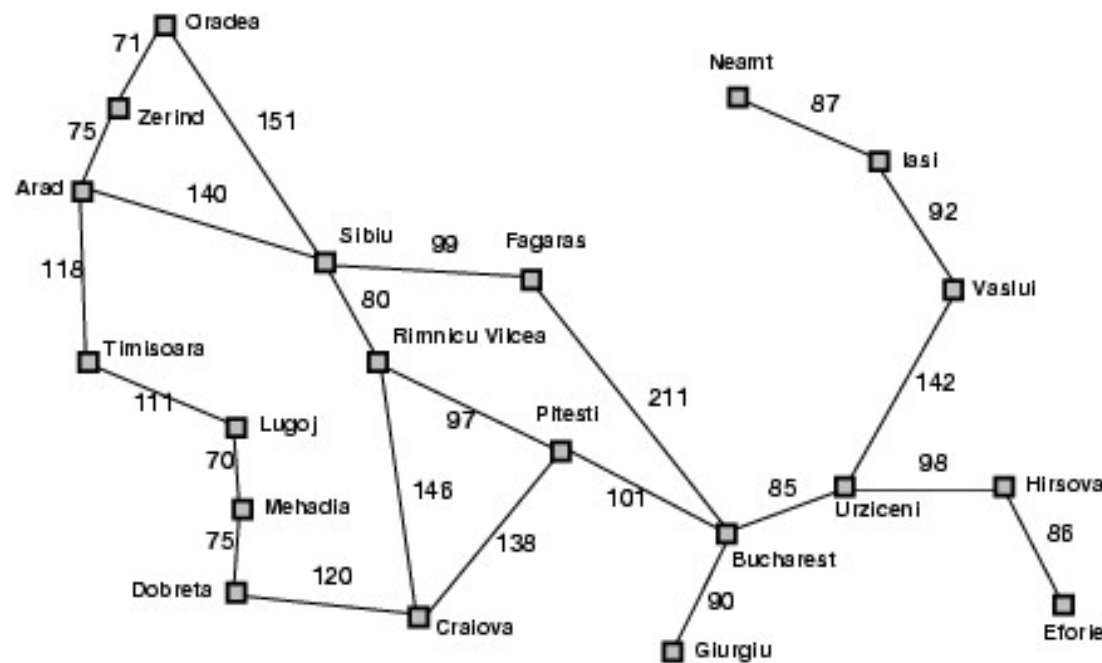
Typical Costs

- $d = 14$, IDS = 3,473,941 nodes
 $A^*(h_1) = 539$ nodes
 $A^*(h_2) = 113$ nodes
- $d = 24$ IDS $\sim 54,000,000,000$ nodes
 $A(h_1) = 39,135$ nodes
 $A(h_2) = 1,641$ nodes

Remark: Given h_1 and h_2 any two admissible functions
then $h(n) = \max \{h_1(n), h_2(n)\}$ is also admissible

A* search example

Arad
366=0+366

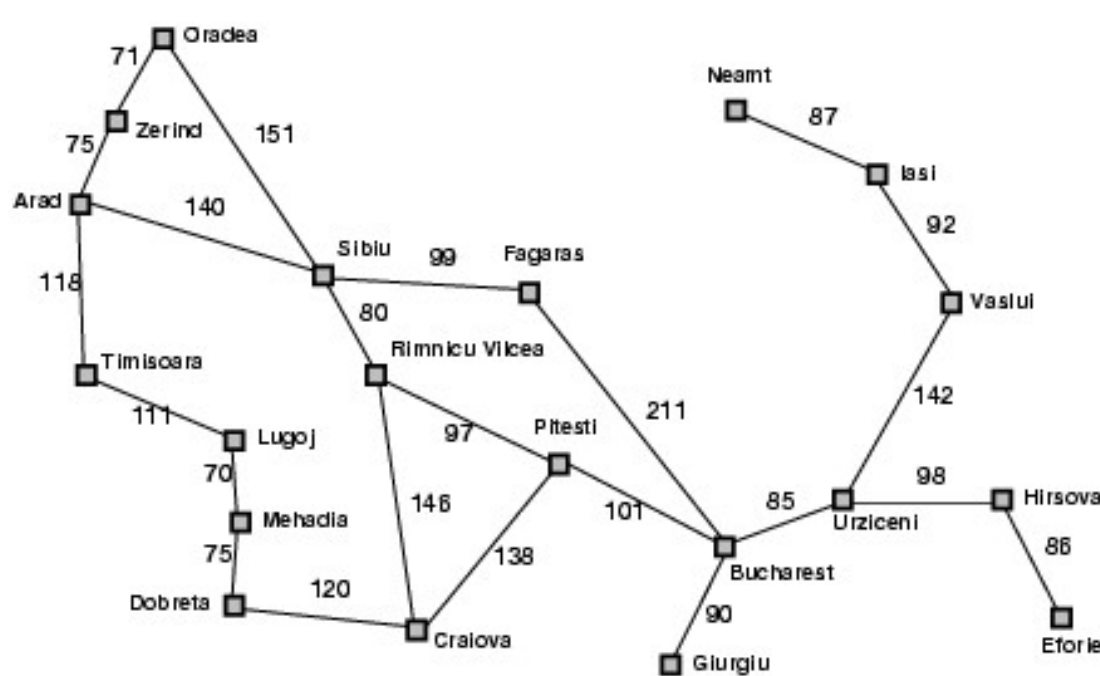
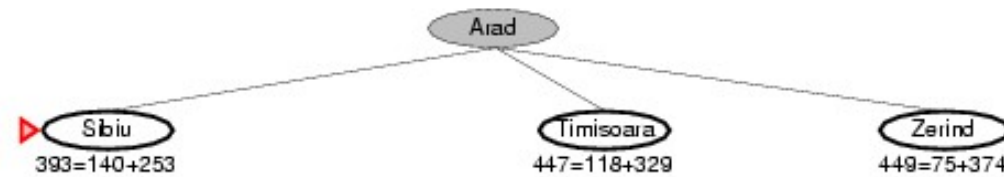


Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	239
Urziceni	98
Vaslui	142
Zerind	374



A* search example

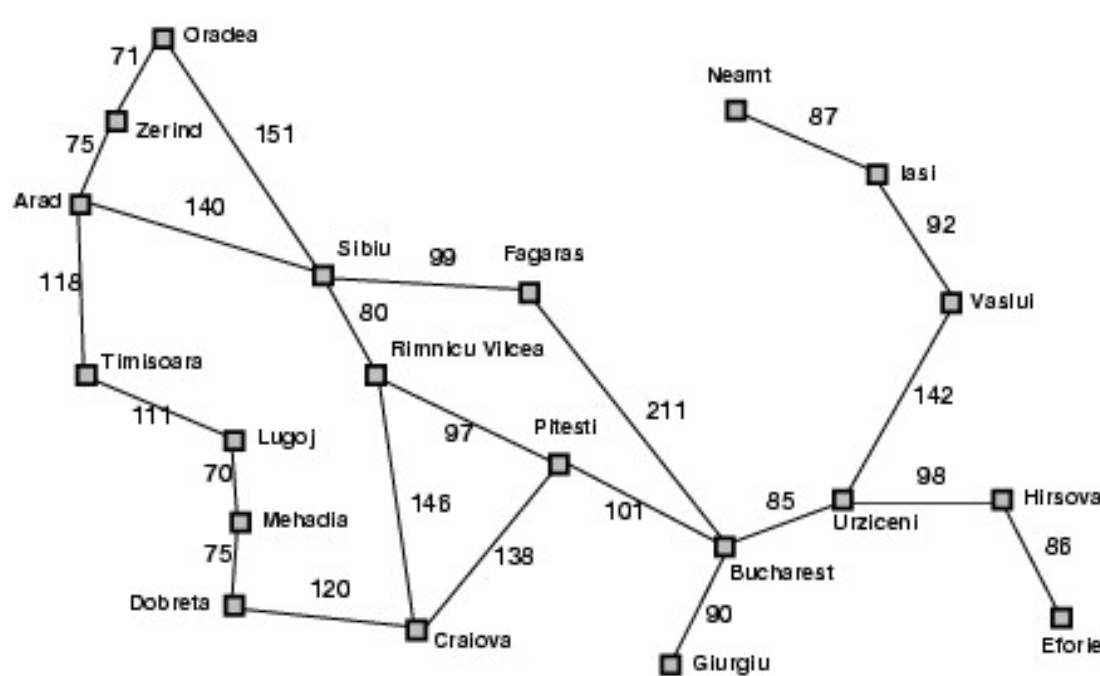
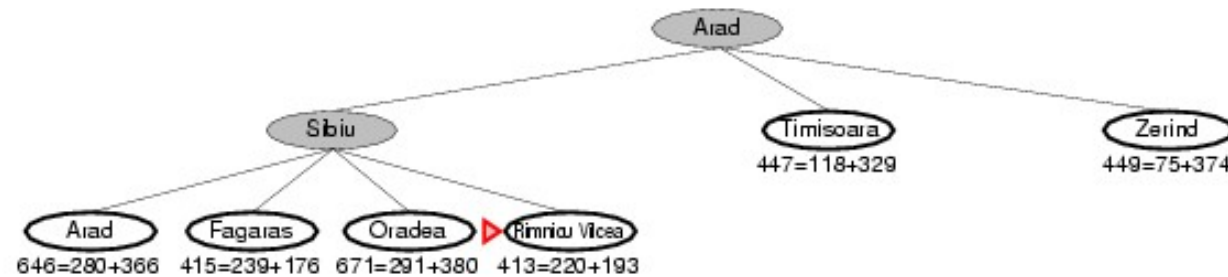


Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	98
Vaslui	191
Zerind	374



A* search example

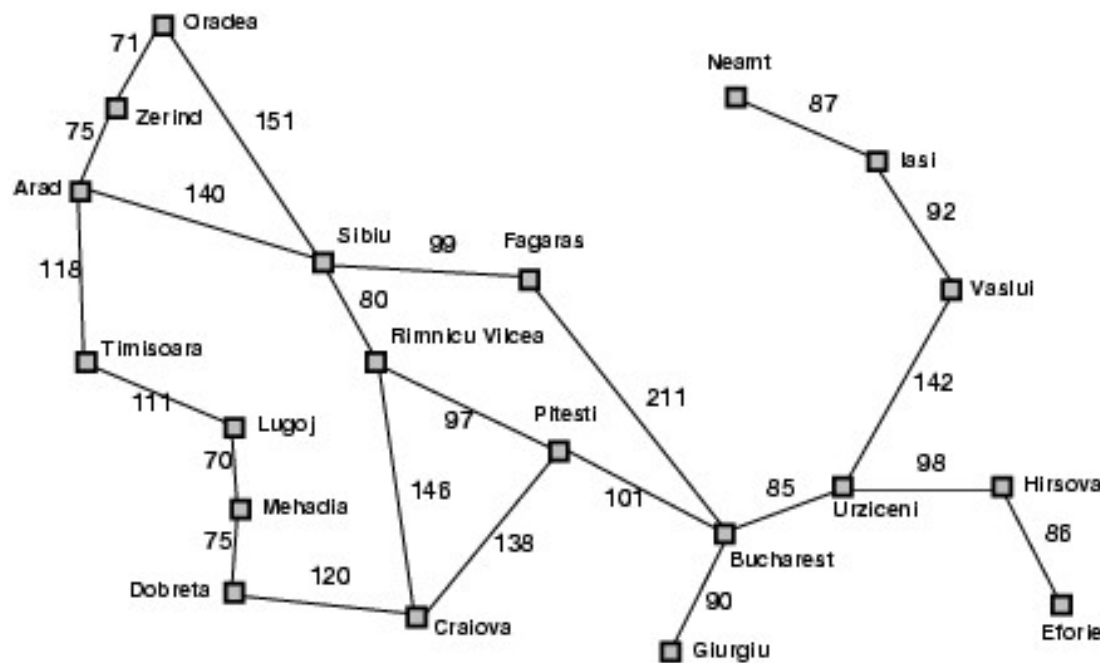
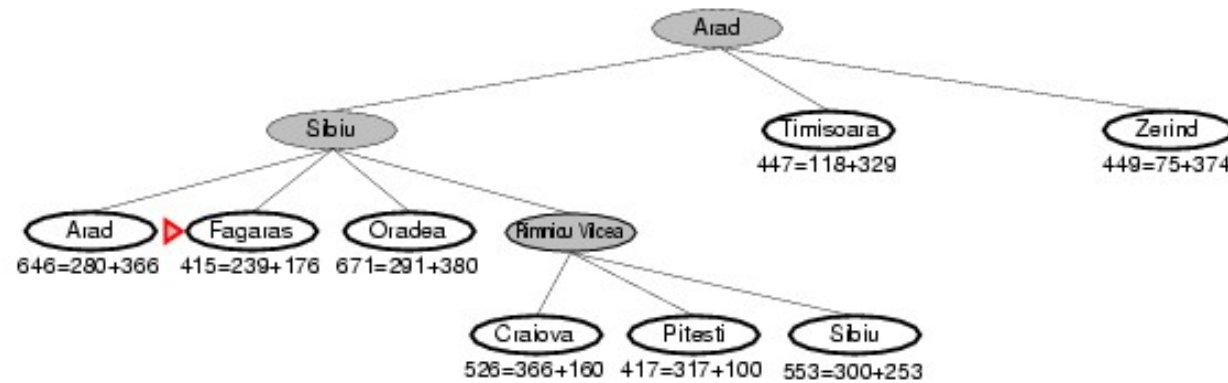


Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	98
Vaslui	142
Zerind	374



A* search example

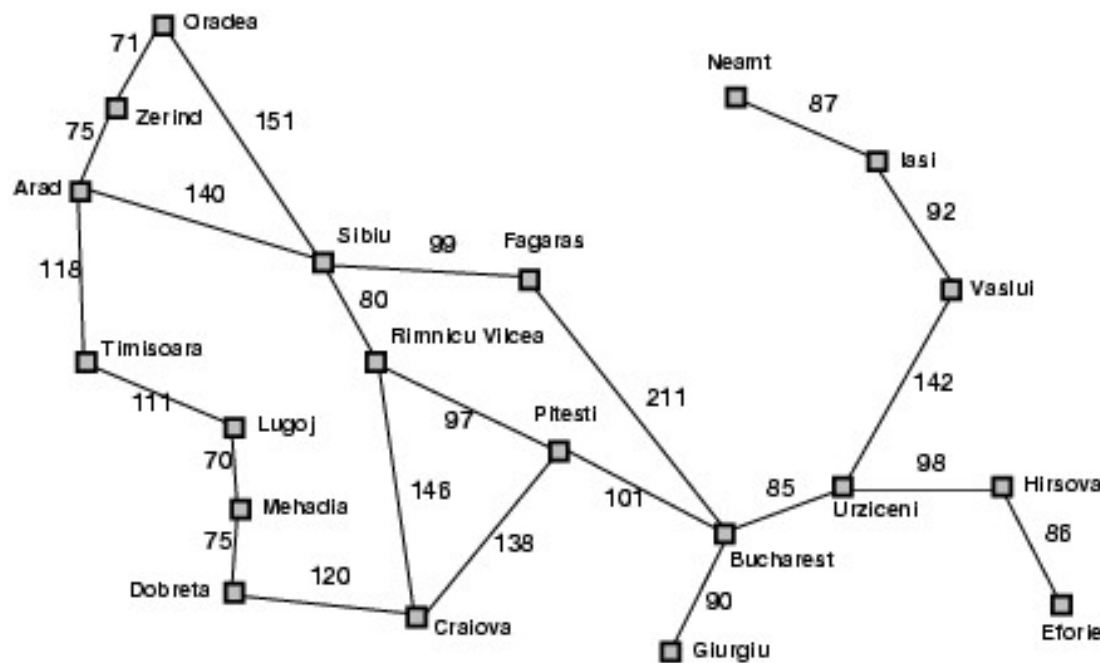
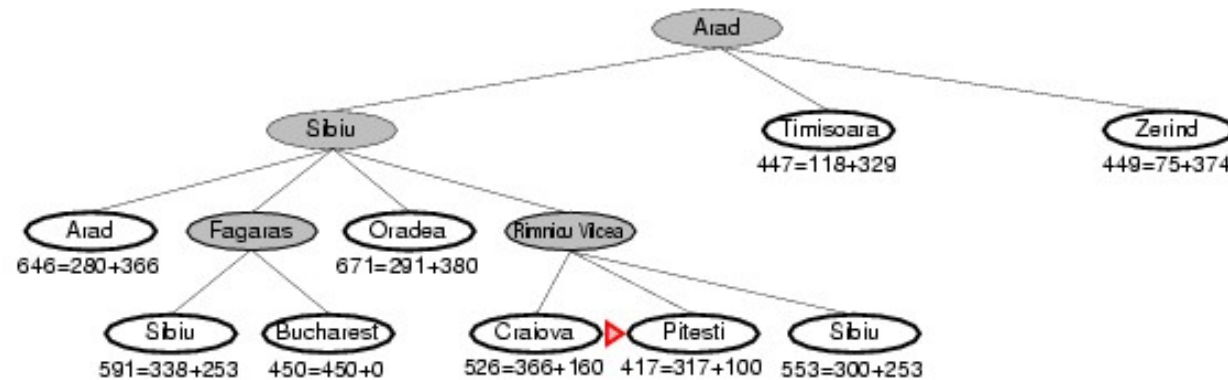


Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	98
Vaslui	142
Zerind	374



A* search example

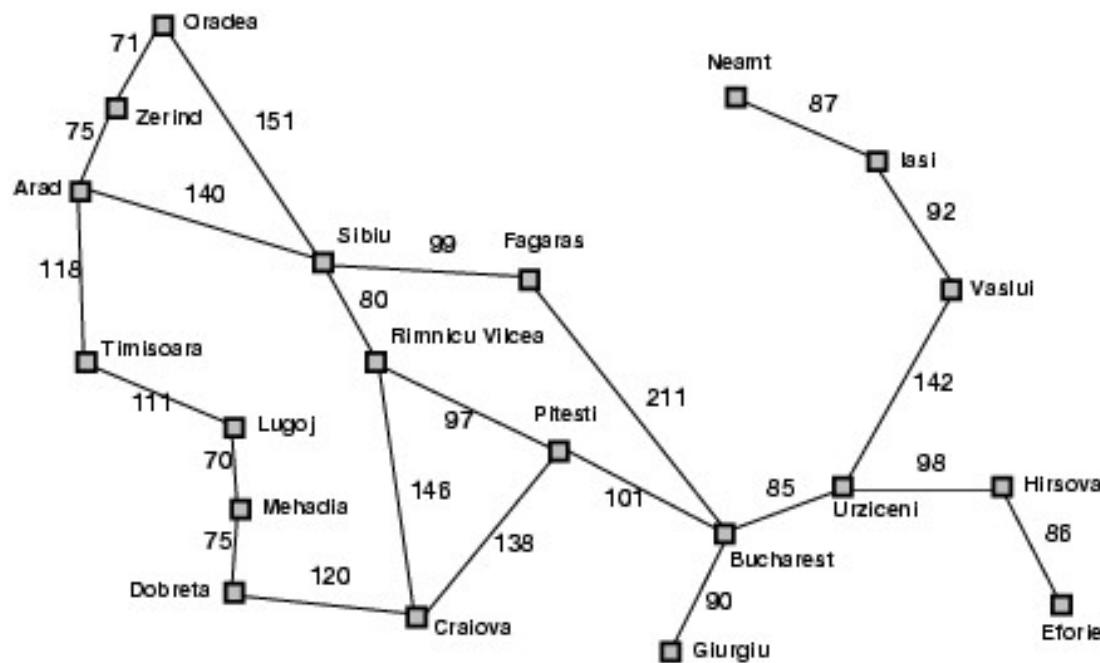
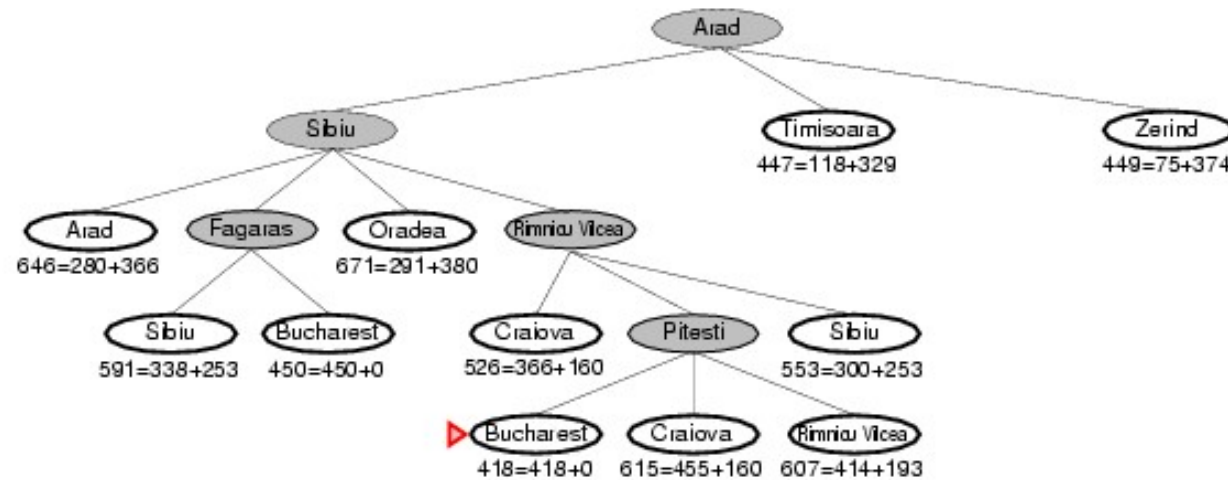


Straight-line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	85
Vaslui	142
Zerind	374



A* search example



Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	142
Zerind	374



Properties of A*

- Complete?

Yes (unless there are infinitely many)

- Time/Space?

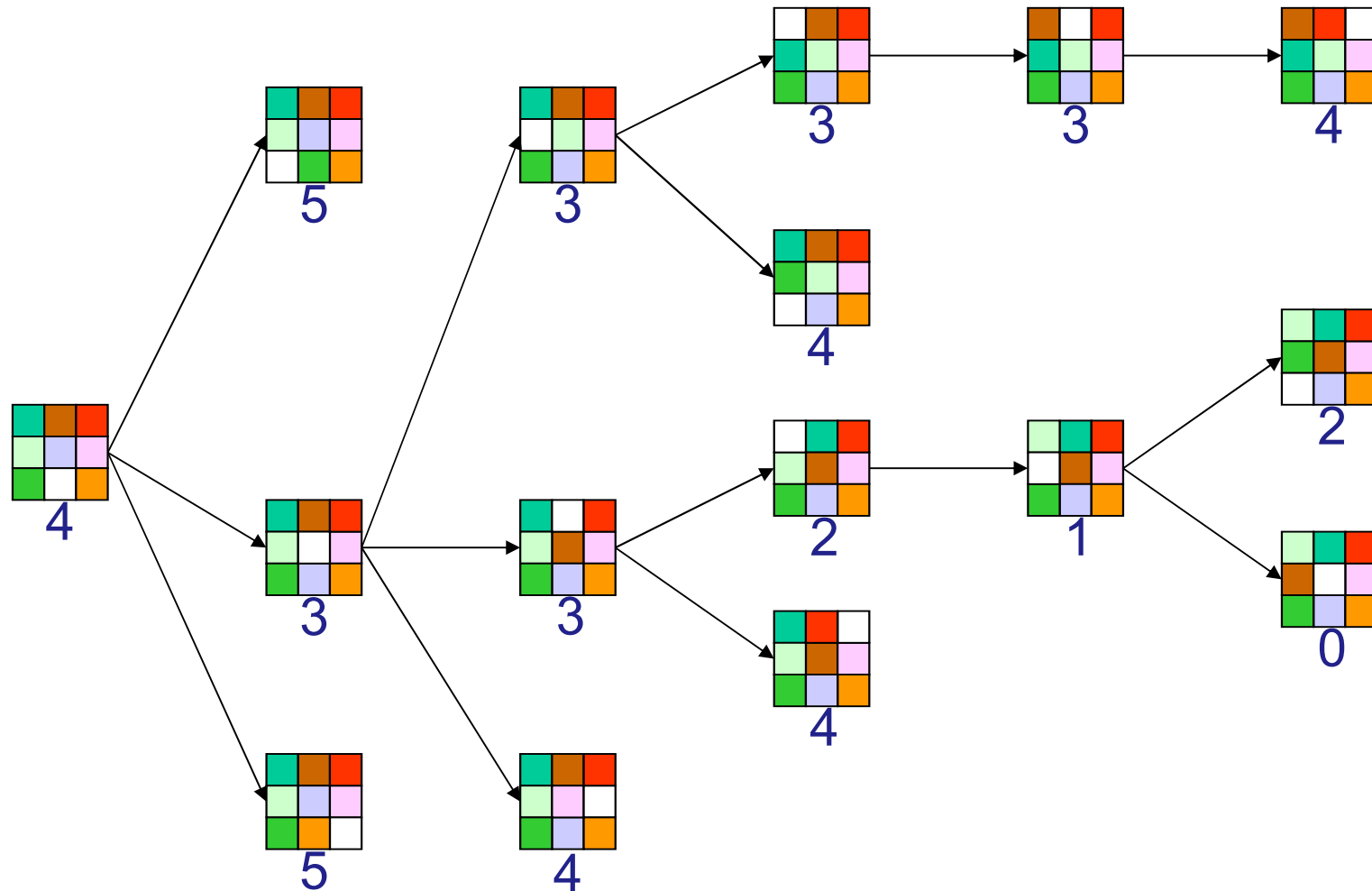
- Exponential mostly

- Optimal?

Yes

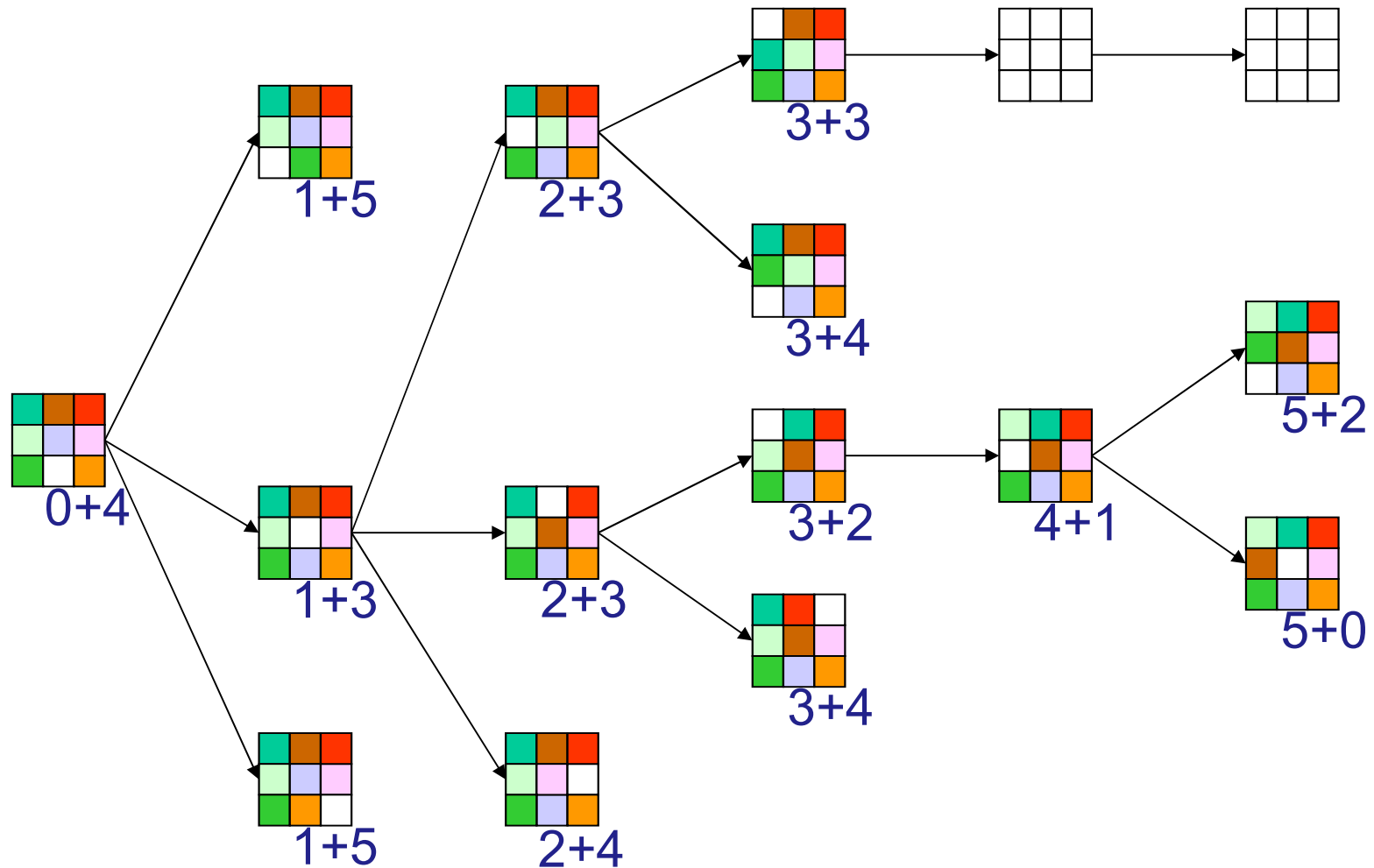
8-Puzzle

$f(N) = h(N) = \text{number of misplaced tiles}$



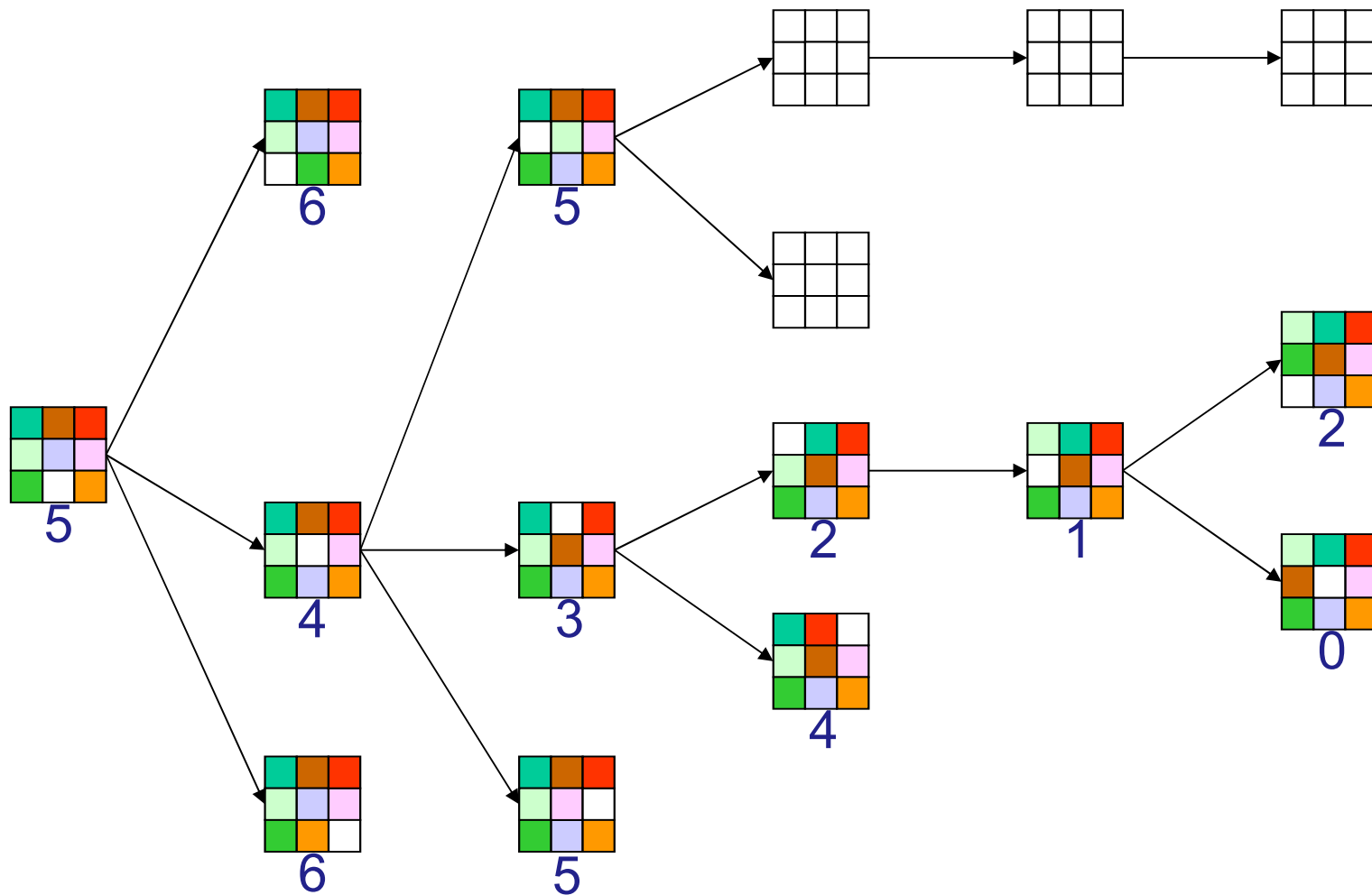
8-Puzzle

$f(N) = g(N) + h(N)$
with $h(N)$ = number of misplaced tiles



8-Puzzle

$$f(N) = h(N) = \sum \text{distances of tiles to goal}$$



Local Search Algorithms

- In many optimization problems, *path* is irrelevant
- the goal state itself is the solution
- Ex: The 8-queen problem, the final configuration of the queens is the important not the order they were put
- Operates using only single current state, rather than multiple paths.
- Find Optimal Configuration (satisfies the constraints)
- Use *iterative improvement algorithms*
- Good for **Optimization** problems: find the best state according to some objective function
- A **Complete** local search algorithm finds a goal if exists
- An **Optimal** algorithm finds the global minimum or maximum

Local Search Algorithms

- Search algorithms like BFS, DFS or A* explore all the search space systematically by keeping one or more paths in memory and by recording which alternatives have been explored.
- Local search algorithms operate using a single current state (rather than multiple paths)
- move only to neighbours of that state.
- Ignore paths
- **Advantages:**
 - Use very little memory
 - Can often find reasonable solutions in large or infinite (continuous) state spaces.

Local Search Algorithms

local search algorithms :

- are useful for solving pure **optimization problems**, in which the aim is to find the best state according to an **objective function**.
- operate using a **single current state** (rather than multiple paths) and generally move only to neighbors of that state.

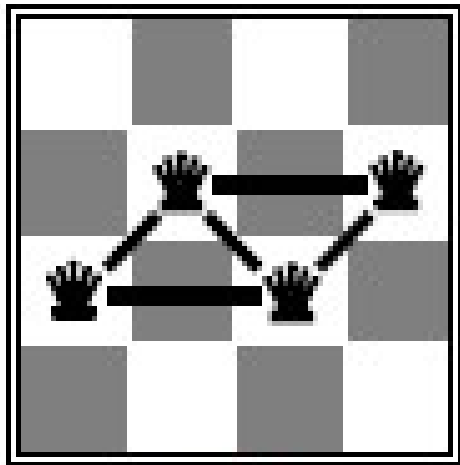
Local Search Algorithms

- Local Search Algorithms are used in:
 - Optimization problems
 - find the best state according to some objective function
 - All states have an objective function
 - Goal is to find state with max (or min) objective value
 - Local search can do very well on these problems.

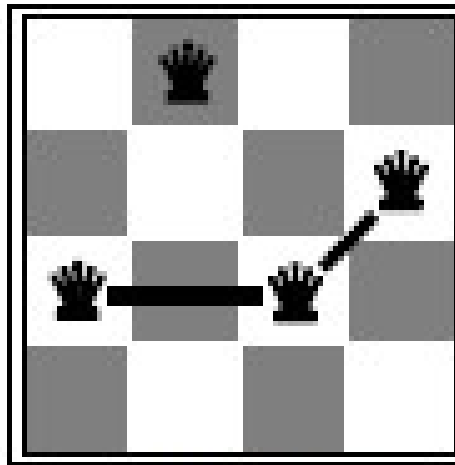
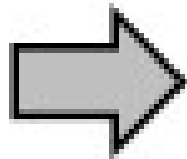
Local Search Algorithms

Example n-queens

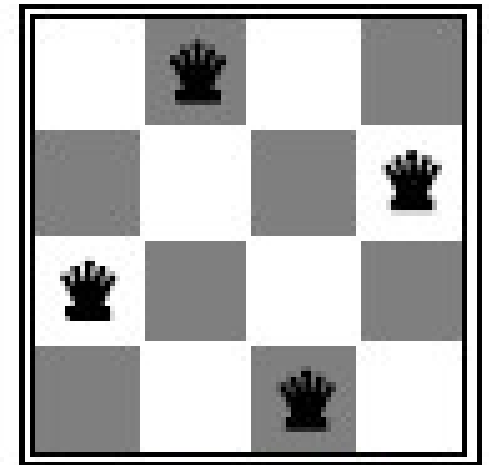
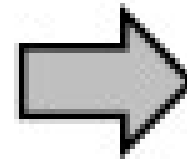
- Put n queens on an $n \times n$ board with no two queens on the same row, column, or diagonal
- Local search: start with all n , move a queen to reduce conflicts



$h = 5$



$h = 2$



$h = 0$

Local Search Algorithms

- Hill Climbing
- Simulated annealing
- Genetic algorithms
- Local search in continuous spaces

Local beam search

- One Solution to improve hill Climbing.
- Keep track of k states instead of one
 - Initially: k randomly selected states
 - Next: determine all successors of k states
 - If any of successors is goal $\rightarrow$ finished
 - Else select k best from successors and repeat.
- Major difference with random-restart search
 - Information is used for k search branches.
- Also improved to stochastic beam search